

DataLab Software Framework and Module Overview

1 Overall Design

DataLab targets high-precision numerical data processing and provides Extrapolation, Uncertainty Propagation, Fitting, and Statistics. The implementation follows a layered structure: an interactive UI layer + a shared core computation layer + a formatting/reporting layer. The desktop app (PySide6) and the Web app (Flask) share the same core computation and the same expression parser/function whitelist, ensuring consistent numerical results, error messages, and supported-function hints across entry points.

1.1 System architecture breakdown

1.2 Function and sub-function trees

From a user workflow perspective, the four major functions (Extrapolation, Error propagation, Fitting, Statistics) share a consistent pipeline: input $\rightarrow$ configuration $\rightarrow$ execution $\rightarrow$ output/export. The differences are mainly in: Extrapolation focusing on method selection and parameters; Error propagation focusing on formula + constants + order/sampling; Fitting focusing on column selection + model/parameter configuration; Statistics focusing on column selection + batching/variance options. To avoid overcrowding, Figures ??-?? show the sub-function trees separately.

1.3 Module responsibilities

1.3.1 Entry points and UI layer

The desktop and Web entry points serve different scenarios: the desktop app focuses on interactive processing, browsing results, and local PDF preview; the Web app focuses on lightweight deployment, cross-platform access, and online export. Neither UI implements algorithm details directly. Instead, both are responsible for:

- collecting user inputs (data/constants/formulas/method parameters/precision settings) and constructing computation jobs;
- calling the shared core computation layer and receiving a unified result payload and warnings;
- presenting outputs as “value + uncertainty + LaTeX + optional plots/files” and supporting exports.

1.3.2 Input and parsing layer

The parsing layer converts text/files into structured data and performs strict validation to avoid implicit downstream errors:

- table parsing: detect headers, convert rows to high-precision numbers (`mp.mpf`), and report missing columns/invalid characters/blank lines;
- uncertainty parsing: support both `value(unc)` notation and split-column inputs, normalizing into (nominal value, standard uncertainty);
- constants parsing: support file and manual inputs, building a constant namespace for expression evaluation and model fitting.

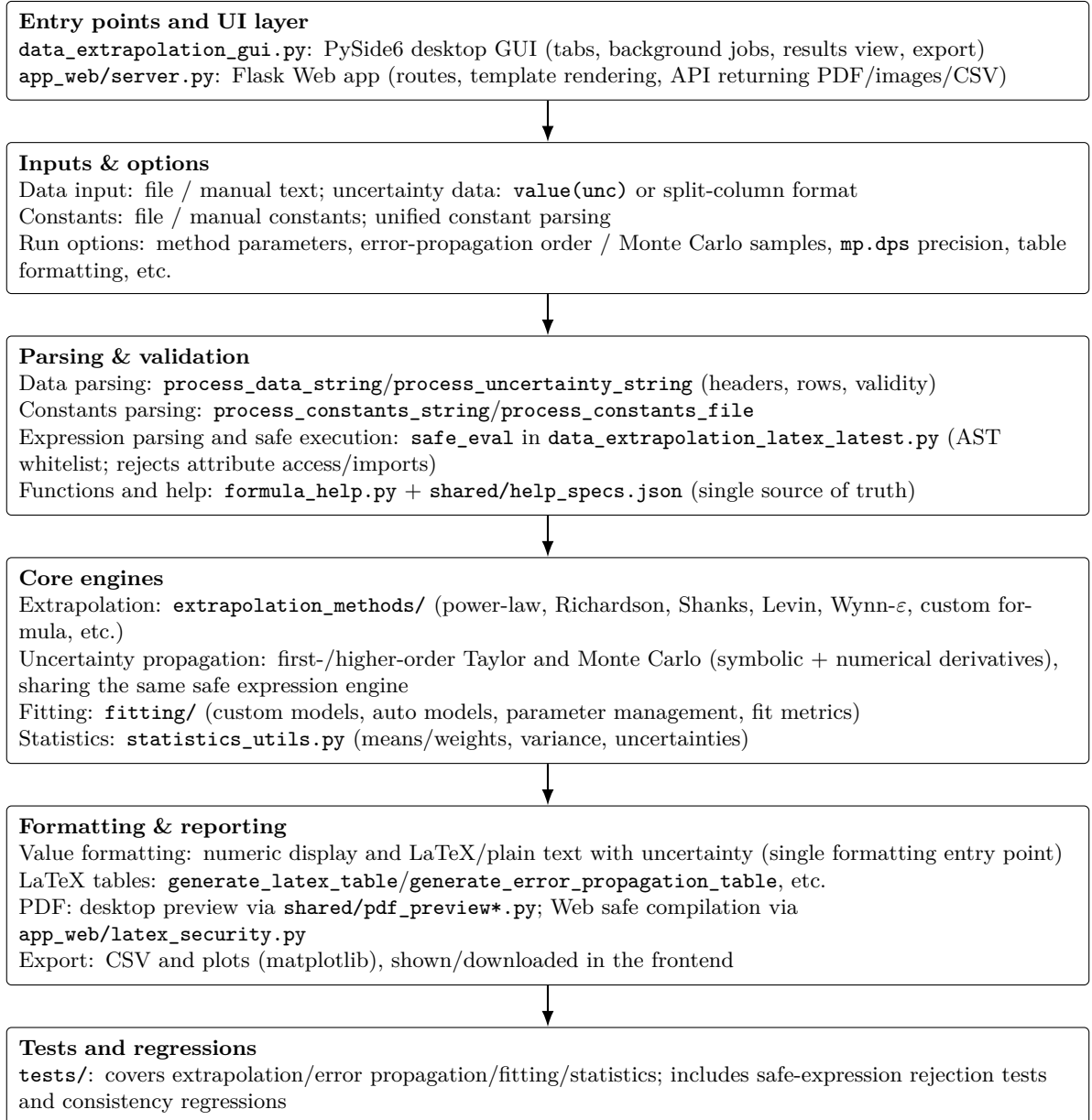


Figure 1: DataLab source code architecture and major module relationships (width constrained by the A4 text width)

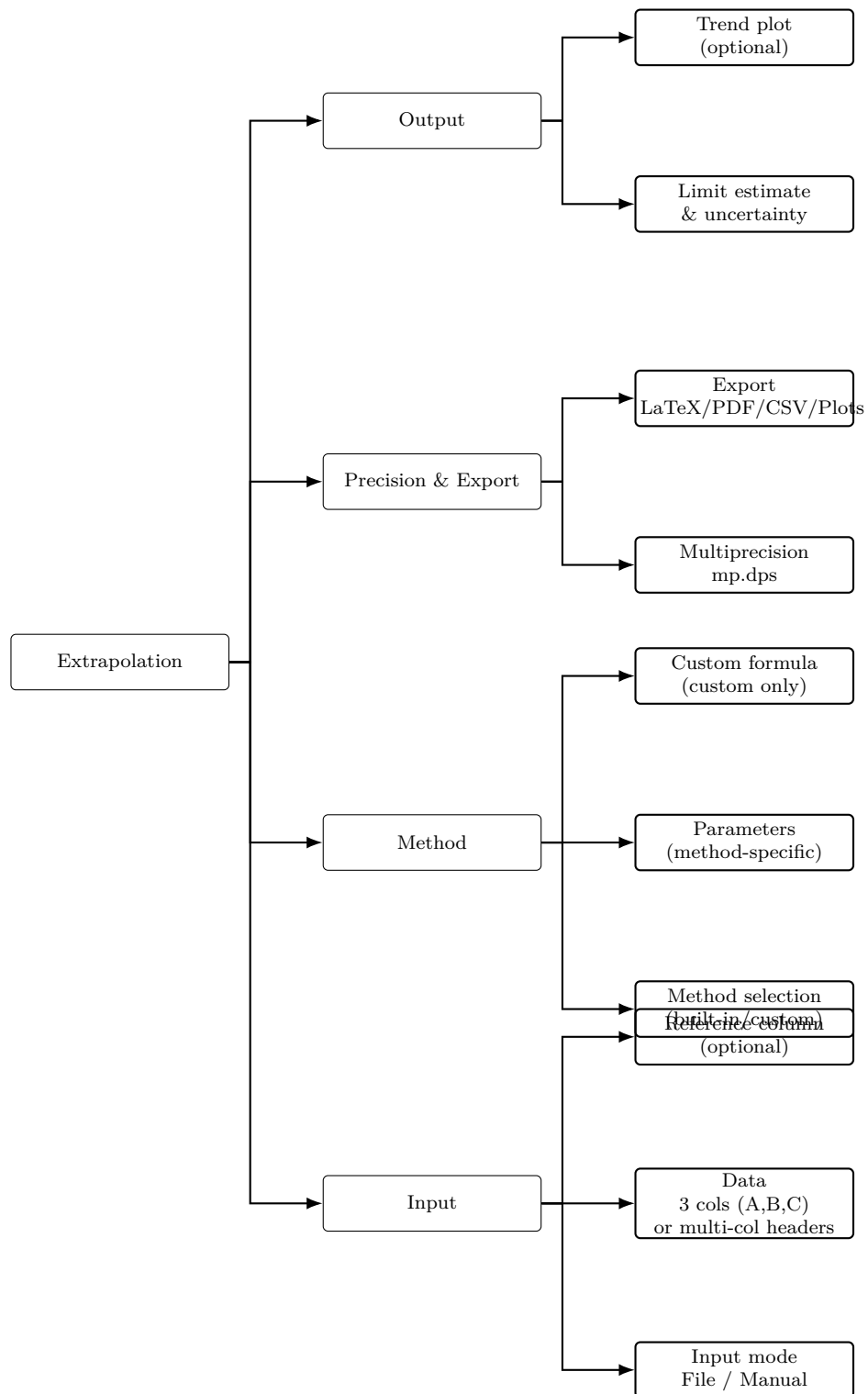


Figure 2: Sub-function tree of the extrapolation module

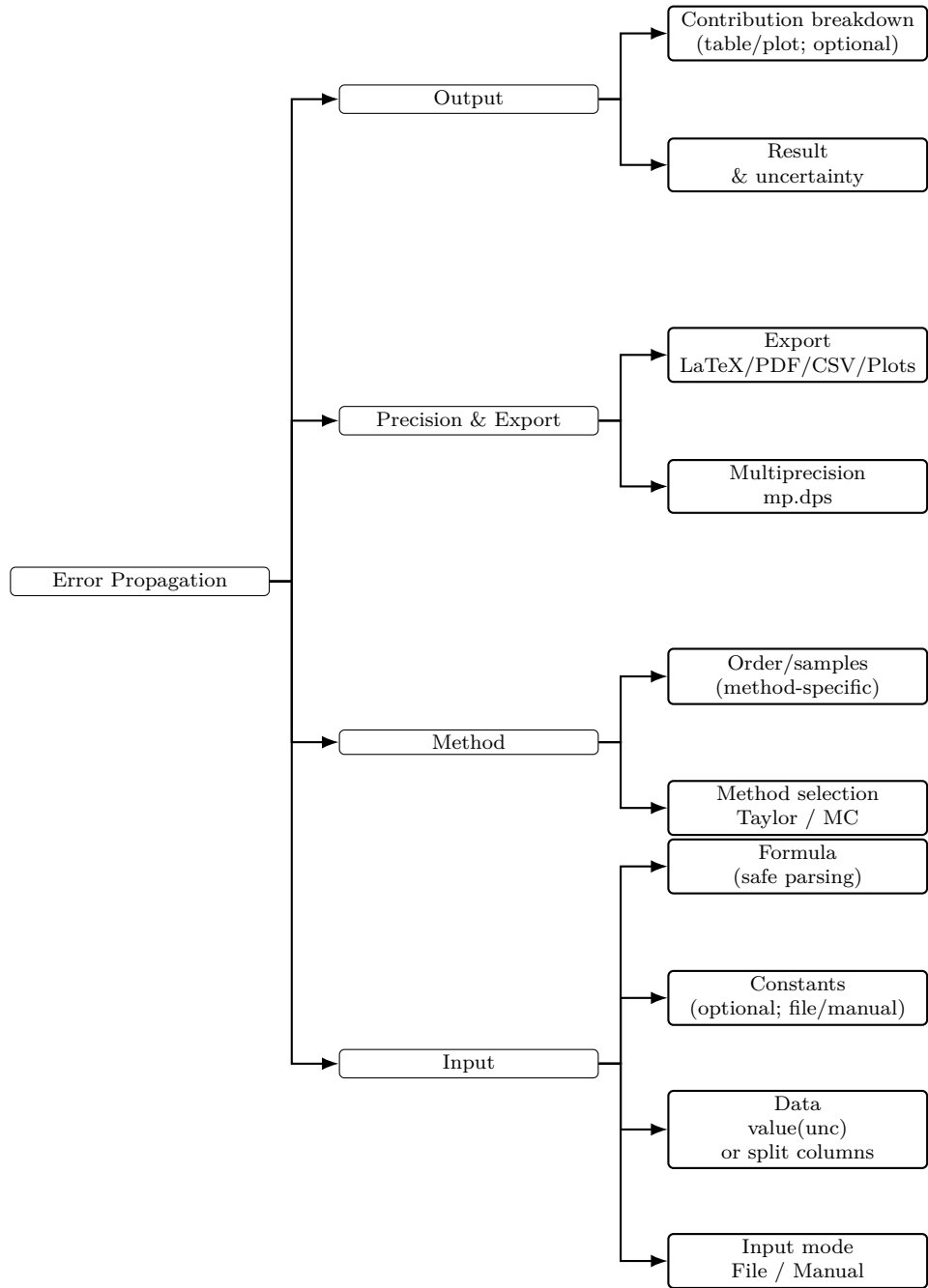


Figure 3: Sub-function tree of the error propagation module

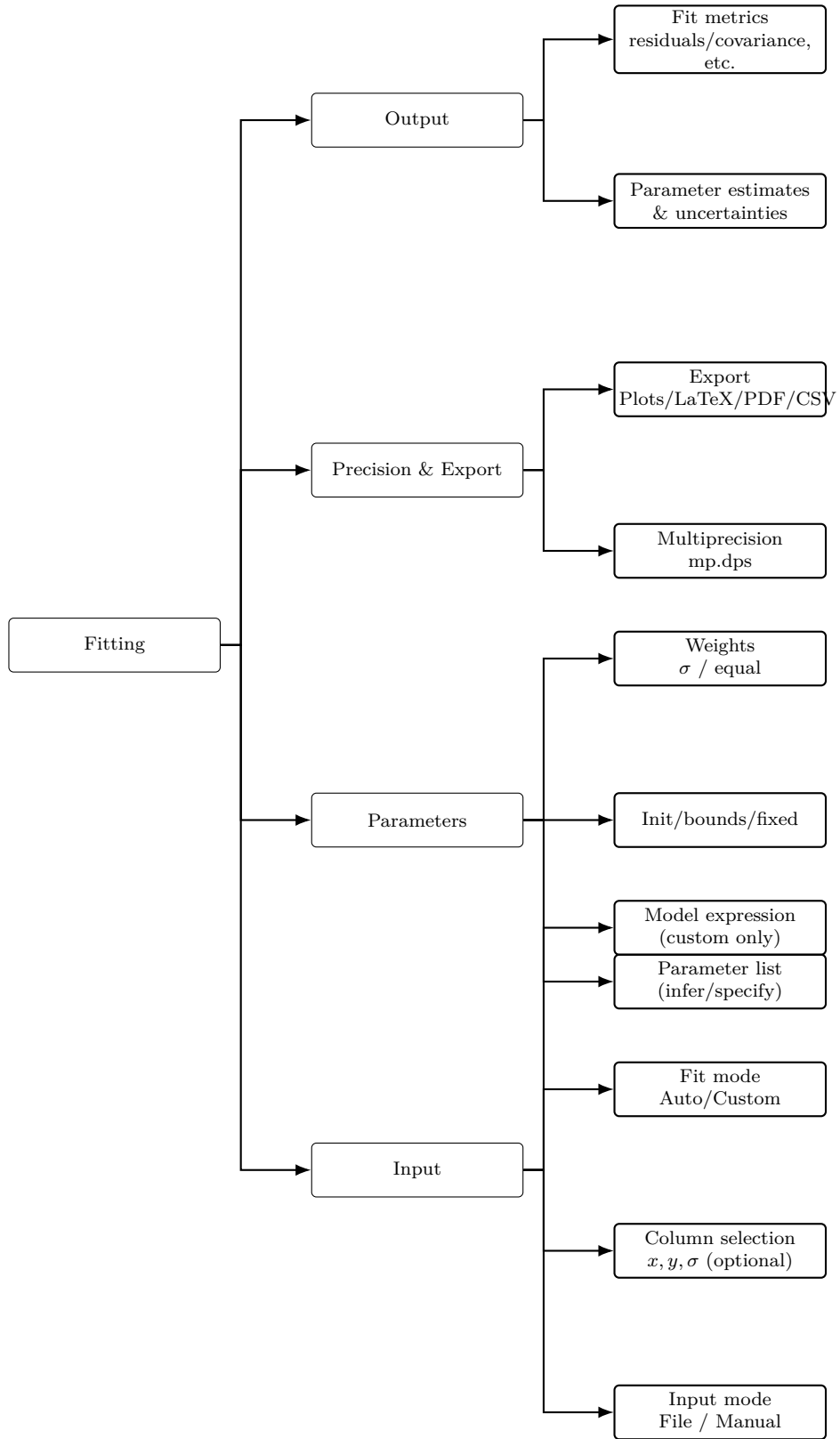


Figure 4: Sub-function tree of the fitting module

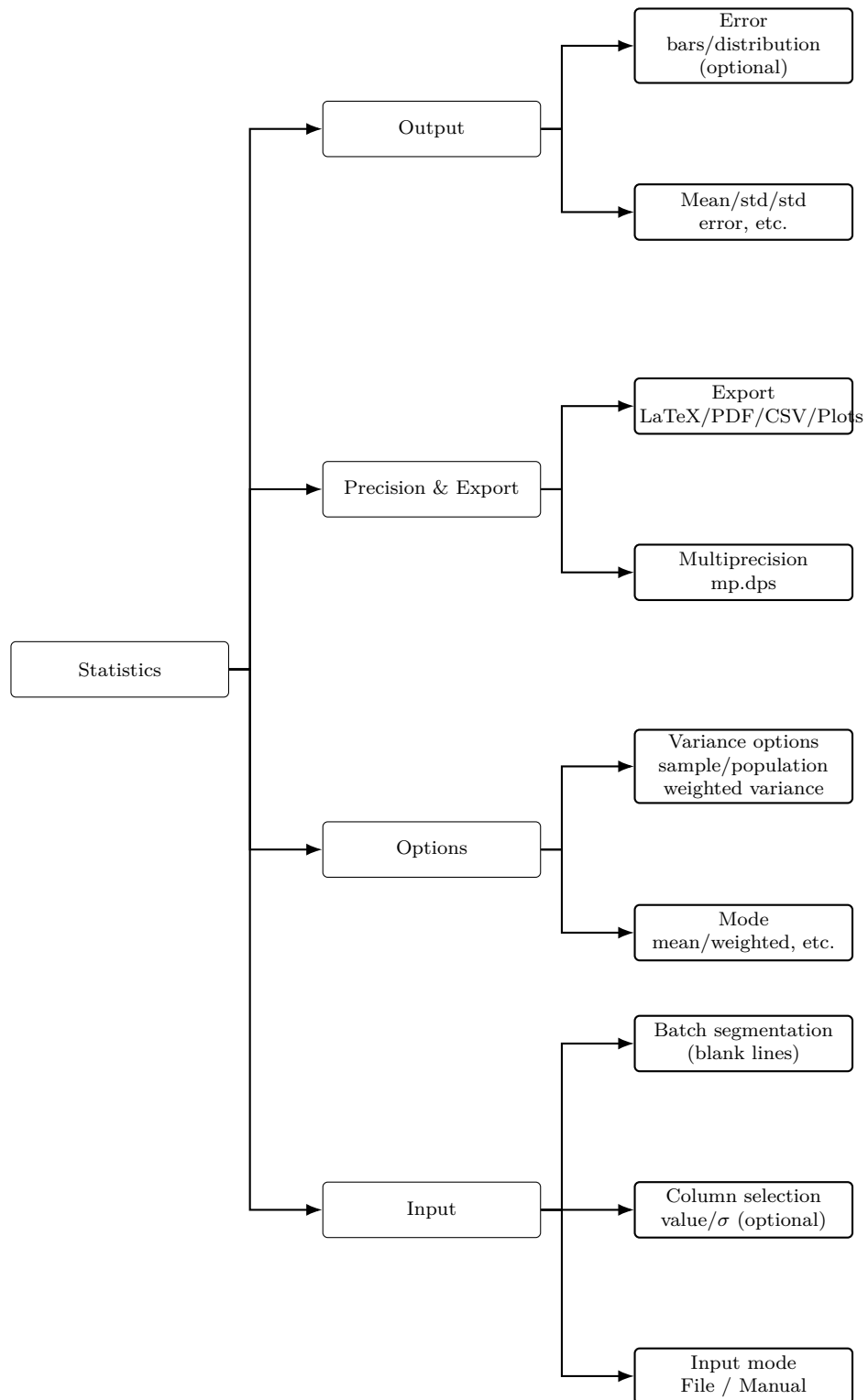


Figure 5: Sub-function tree of the statistics module

1.3.3 Expression parsing and safe evaluation

Custom extrapolation formulas, error-propagation formulas, and custom fitting models share the same safe expression engine. The core idea is to parse expressions with an AST and allow only whitelisted node types, function calls, and constants, while explicitly rejecting attribute access, imports, arbitrary object construction, and other dangerous behaviors. This design provides:

- security: avoids code execution risks of `eval/exec`;
- consistency: all formula inputs share the same function/constant registry and error-message style;
- extensibility: adding a function to the whitelist enables it across all formula-based scenarios and keeps help/hints in sync.

1.3.4 Core engines

The core engines implement the four classes of computations and use a unified precision control (`mp.dps`) for numerical stability:

- extrapolation: multiple acceleration/extrapolation methods plus custom three-point formulas, producing a limit estimate and uncertainty;
- uncertainty propagation: first-order, higher-order Taylor contributions, and Monte Carlo sampling; outputs values, uncertainties, and optional contribution breakdowns;
- fitting: custom and auto models, producing parameter estimates, fitted curves, and metrics (e.g., covariance matrices and parameter uncertainties);
- statistics: batch-aware and weighted statistics, producing mean/variance/standard error and related indicators.

1.3.5 Formatting and reporting

The output layer formats results for readability and reuse, supporting both on-screen display and LaTeX export while keeping precision and formatting rules consistent:

- uncertainty formatting: a single LaTeX/plain-text formatting entry point to avoid cross-module inconsistencies;
- LaTeX tables: unified table environments, numeric column specs (`dcolumn/siunitx`), segmentation, etc.;
- PDF: desktop preview and zoom; Web safe compilation (engine whitelist, shell-escape disabled, resource limits).

1.3.6 Shared specifications and consistency guarantees

To ensure “same computation, same behavior” across UIs, UI-facing specifications (supported functions, method descriptions, parameter visibility) are maintained centrally. For example, `formula_help.py` and `shared/help_specs.json` provide the unified function list and help text, and `shared/ui_specs.py` describes method parameters and display rules. Frontends render these specifications rather than hardcoding function lists or duplicating specs.

1.3.7 Tests and regressions

The project uses `pytest` to maintain regression tests covering correctness, consistency, and security. Typical checks include cross-module consistency for representative formulas and special functions; rejection tests for dangerous expressions; and targeted unit tests with numeric tolerances for newer features (higher-order propagation, custom fitting models, etc.).